

# Zarfın Üstündeki Bölmeler

Bilge ve Yonga'nın Maceraları — Buyrukların Dünyası



Prof. Dr. Oğuz Ergin



"Yonga, işlemci bir buyruğu aldığıında ne yapacağını nasıl anlıyor?" diye sordu Bilge. "Buyruğun şekline bakarak!" dedi Yonga. "Her buyruk aslında 32 tane sıfır ve birden oluşur. Bu bitleri belirli gruplara böldüğünde anlam çıkar, tıpkı bir zarfın üzerindeki bölmeler gibi: ad, adres, posta kodu."





"Her buyruğun en önemli bölümü işlem kodudur." dedi Yonga. (opcode: ne tür işlem yapılacağını söyler)  
"İşlem kodu, zarfın ilk satırı gibi: 'Bu ne tür bir mektup? Fatura mı, davetiye mi, haber mi?' İşlemci işlem koduna bakarak 'Bu bir toplama buyruğu' ya da 'Bu bir bellek okuma buyruğu' diye anlar."





"Neden tek bir biçim yetmiyor?" diye sordu Bilge. "Gönderinin türü değişince zarf da değişir." dedi Yonga. "Bir kartpostalda yalnız adres vardır. Bir kargoda ağırlık, boy, gönderen de yazar. Buyruklar da öyle: kimi üç yazmaç ister, kimi bir sayı taşır, kimi gidilecek yeri söyler. Bu yüzden altı zarf biçimi var. Her biri farklı bilgiler taşır ama hepsi 32 bit boyundadır." "Zarflar farklı ama hepsi aynı posta kutusuna sığıyor!" dedi Bilge.





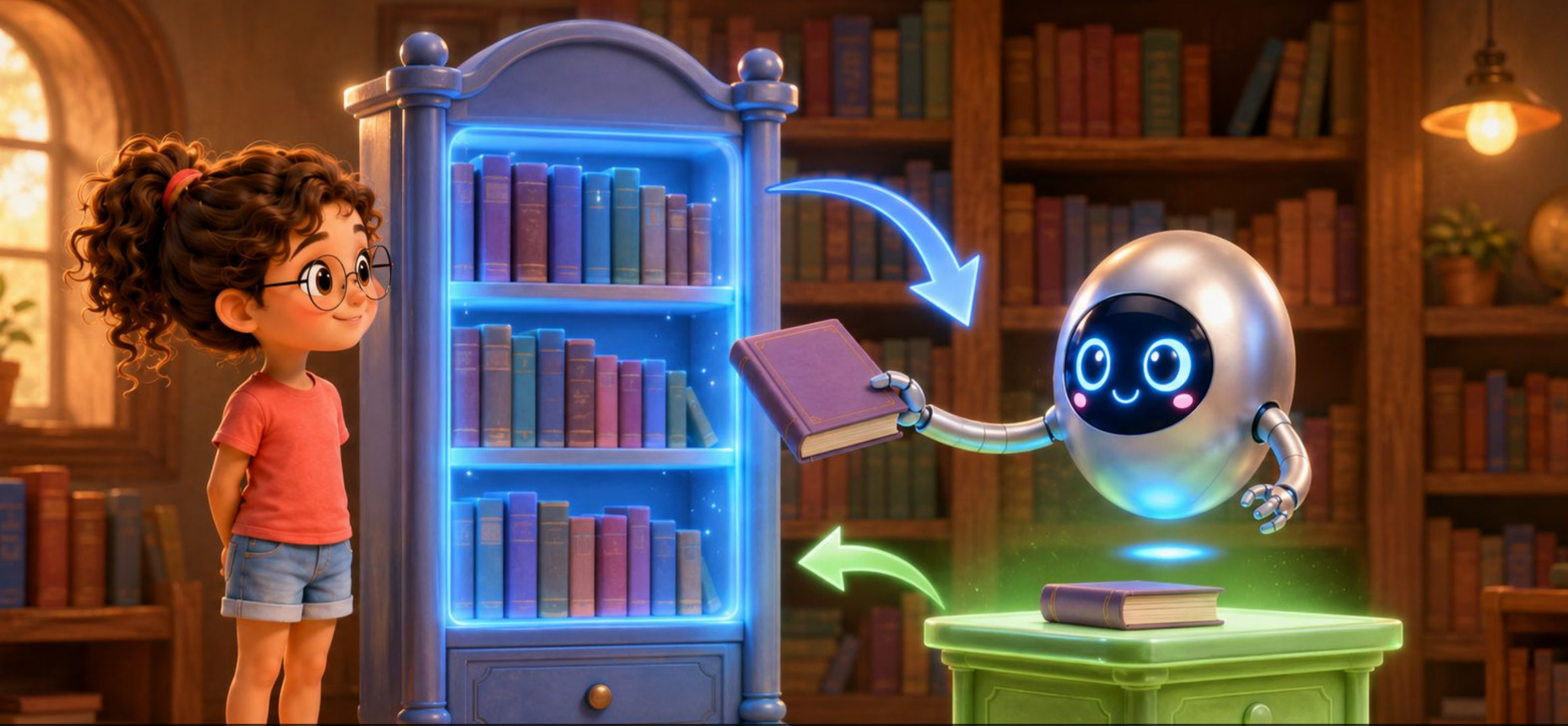
"RISC-V'te altı farklı buyruk biçimi var." dedi Yonga. "İlki R biçimi: 'Register' sözcüğü yazmaç anlamına gelir. R biçiminde üç yazmaç alanı var: • rs1: Birinci kaynak yazmaç • rs2: İkinci kaynak yazmaç • rd: Hedef yazmaç (sonuç buraya gider) Örneğin: 'x5 + x6 → x7' tam bir R biçimi buyruğu!" "Bunu nasıl yazarız?" diye sordu Bilge. "Mühendisler kısaca ADD x7, x5, x6 yazar. Kural şu: en başta hedef yazmaç durur, arkasından iki kaynak gelir. Sonucun gideceği yer hep en öndedir!"





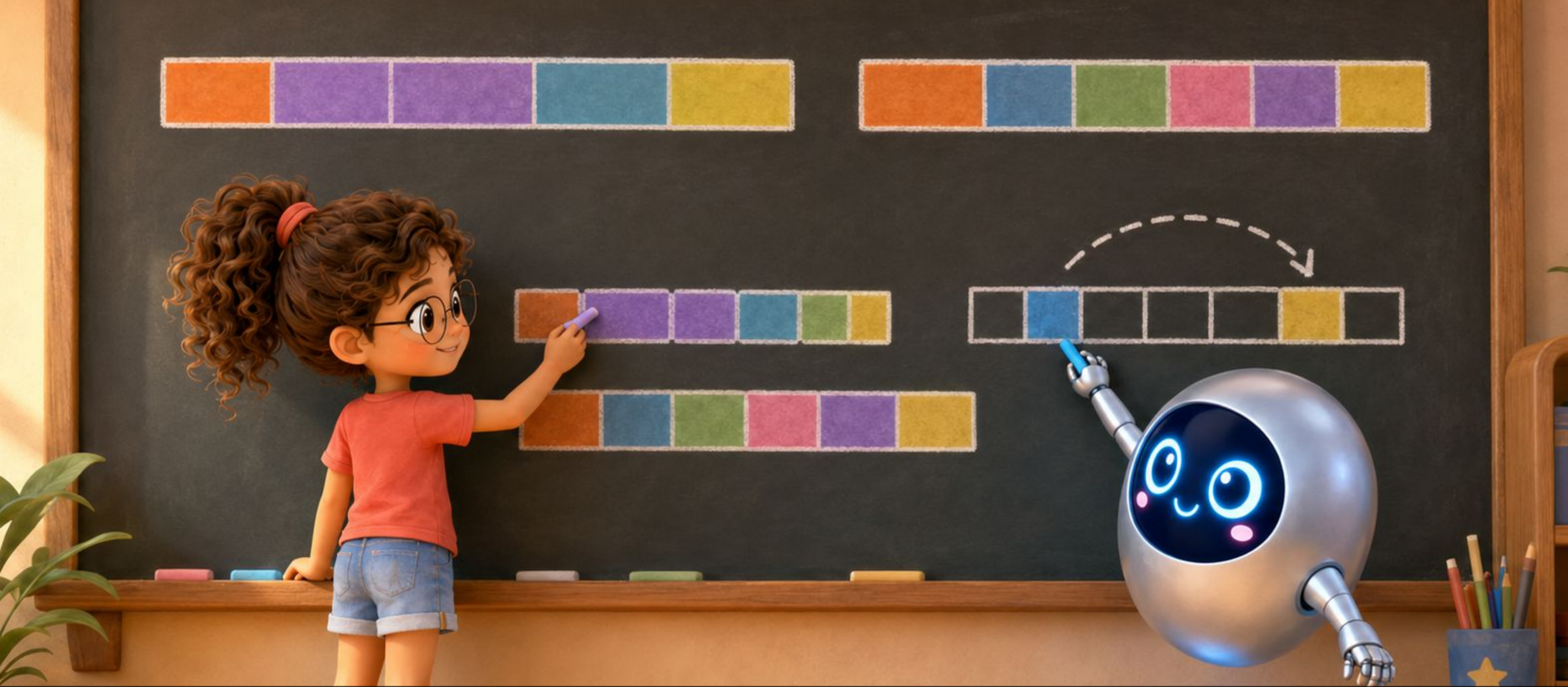
"I biçimi biraz farklı." dedi Yonga. "'I' harfi İngilizce 'immediate' sözcüğünden gelir, anlık demektir. Sayı buyruğun içinde gömülüdür. Örneğin ' $x5 + 5 \rightarrow x6$ ' buyruğunda 5 sayısı bellekte ya da yazmaçta değil, buyruğun ta kendisinin içindedir!" "Vay canına!" dedi Bilge. "Sayıyı yanında getiriyor." "Tıpkı 'bana 3 tane ver' demek gibi, 3'ü dışarıdan aramana gerek yok, zaten söyledin!"





"Veri aktarma buyrukları iki yönlü çalışır." dedi Yonga. "YÜKLE (load): Bellekten oku, yazmaca koy. Örnek: 'Adres 100'deki sayıyı x5'e yükle.'" "SAKLA (store): Yazmaçtaki değeri belleğe yaz. Örnek: 'x5'teki sayıyı adres 200'e sakla.'" "İkisi birbirinin tersi." dedi Bilge. "Bir kütüphaneden kitap almak ve kitap iade etmek gibi!"





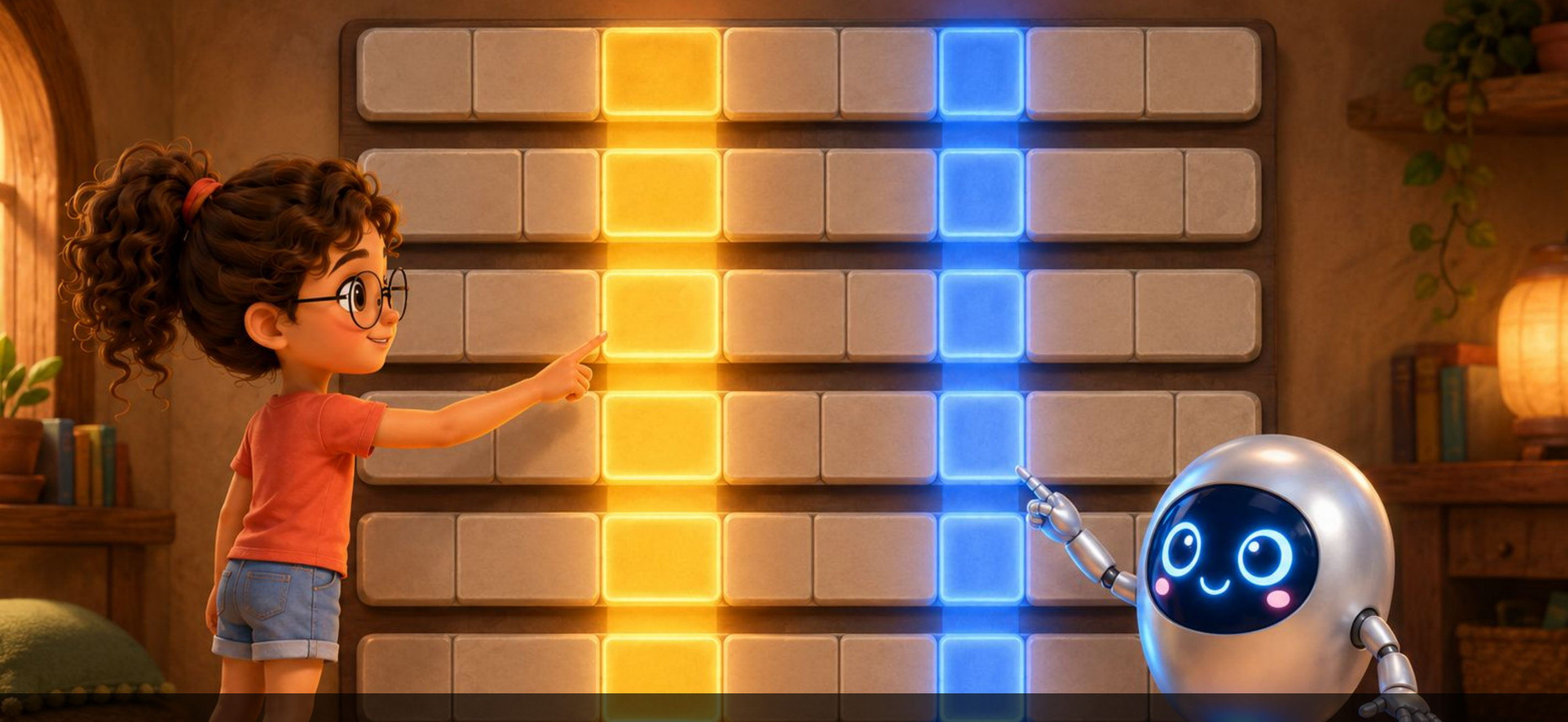
"S biçimi saklama buyrukları içindir." dedi Yonga. "Adresi bulmak için kullanılan kayma değeri buyruğun içinde iki parçaya bölünmüş halde saklanır." "Neden bölünmüş?" diye sordu Bilge. "Çünkü RISC-V'te her şey 32 bite sıkıştırılmıştır. Yer az, akılcıca kullanmak gerekir!" "B biçimi ise dallanma buyrukları içindir." dedi Yonga. "'Koşul doğruysa şu adrese zıpla' demek için kullanılır."





"Sırada son iki biçim var." dedi Yonga. "U biçimi büyük anlık sayılar içindir. Bazen adres ya da büyük değer oluşturmak için gerekir." "J biçimi ise atlama içindir, fonksiyon çağırmak gibi. 'Şu adrese git, işini bitir, geri dön' demek için." "Altı biçim, altı farklı zarf." dedi Bilge. "R, I, S, B, U, J, bunları ezberliyorum!"





"İşlemci hangi zarfın geldiğini nereden biliyor?" diye sordu Bilge. "En akıllı kısmı burada." dedi Yonga. "Altı biçim farklı, ama bazı bölmeler hepsinde aynı yerde durur. Bir zarfta hedef yazmaç varsa hep aynı bitlerde durur; birinci kaynak da öyle. Her zarfta ikisi birden bulunmaz. Böylece işlemci zarfın türünü daha anlamadan o bölmeleri okumaya başlar. Zaman kazanır." "Bakmadan uzanıp alıyor." dedi Bilge. "Kalemin hep aynı gözde durduğunu bildiğin gibi!"





"Peki bir buyruk, bellekteki adresi nasıl gösterir?" dedi Yonga. "Koca bir adres 32 bitlik buyruğa sığmaz. Bu yüzden bir yazmaçtaki adresten yola çıkarız. Buna adresleme kipi denir: Taban + Kayma: 'Şu yazmaçtaki adrese 8 ekle, oraya git.' Yükleme ve saklama böyle çalışır. Yazmaç Dolaylı: Kaymayı sıfır yap, doğrudan yazmaçtaki adrese git. PC Göreceli: 'Şu anki konumdan 20 adres ileriye git.' Dallanmalar böyle çalışır." "Demek adresi baştan değil, yakınından yazıyoruz!" dedi Bilge.





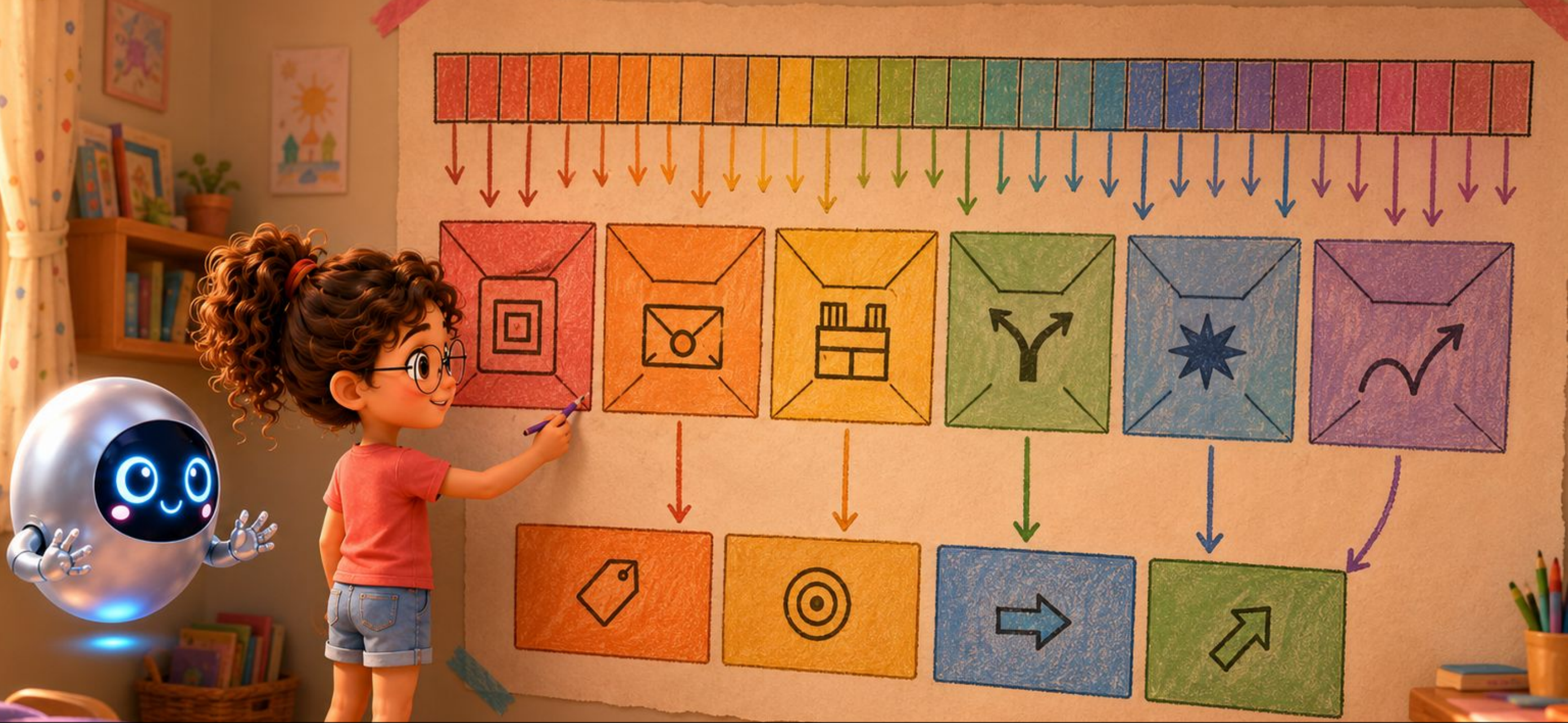
"Hadi bir buyruđu birlikte çözelim." dedi Yonga ve havaya otuz iki kutuluk bir řerit çizdi. "řu buyruk geldi: ADD x7, x5, x6. İşlem kodu bölmesi, 'bu bir yazmaç işlemi' diyor. Öteki bölmelerde hedef x7 ile kaynaklar x5 ve x6 duruyor." "Kalan bölme ne?" diye sordu Bilge. "Hangi işlem olduğunu söylüyor: toplama mı, çıkarma mı. İşlem kodu aileyi seçer, o bölme de tek tek işi."





Bilge ellerini birbirine vurdu: "Anladım! Her buyruk aslında 32 bit. Bu bitlerin biçimi (R, I, S, B, U, J) ne tür buyruk olduğunu söylüyor. İçinde işlem kodu, kaynak yazmaçlar, hedef yazmaç ve bazen anlık sayı var." "Ve işlemci bu şifreyi saniyede milyarlarca kez çözüyor." dedi Yonga. "Müthiş." dedi Bilge. "Her bit bir anlam taşıyor."





O gece Bilge duvara bir şema yapıştırdı: Bir buyruk: 32 bit ↓ Altı zarf biçimi ↓ İşlem kodu Hedef yazmaç Kaynak yazmaçlar Anlık sayı "Zarfın üstündeki her bölme bir soruya yanıt veriyor, ve işlemci hepsini bir bakışta okuyor." dedi Bilge.



# Bugün Ne Öğrendik?

---

- İşlem kodu (opcode): Buyruğun ne tür işlem yapacağını belirten bit grubu.
- R biçimi: İki kaynak yazmaç + bir hedef yazmaç. Örnek: ADD, SUB.
- Buyruk yazımı: Önce hedef yazmaç, sonra kaynaklar. `ADD x7, x5, x6` demek "x5 ile x6'yı topla, sonucu x7'ye yaz" demektir.
- I biçimi: Bir kaynak yazmaç + buyruğun içine gömülü anlık sayı. Örnek: ADDI.
- S biçimi: Saklama buyrukları için. Adresi bulmaya yarayan kayma değeri iki parçada.
- B biçimi: Dallanma buyrukları (koşullu atlama).
- U biçimi: Büyük anlık sayılar için.
- J biçimi: Koşulsuz atlama / fonksiyon çağırısı.
- YÜKLE / SAKLA: Bellekten yazmaca al / yazmaçtan belleğe yaz.
- Altı biçim, tek boy: Zarflar farklı bilgiler taşır ama hepsi 32 bittir.
- Alanlar hep aynı yerde: Bir alan kullanılıyorsa yeri biçimden biçime değişmez; işlemci buyruğun türünü anlamadan onları okumaya başlar. Her biçimde her alan bulunmaz.
- Adresleme kipleri: Bir bellek adresini farklı şekillerde belirtme yolları.



# Deneme Zamanı

---

1. Buyruğun ne tür işlem yapacağını belirten bit grubunun adı nedir?
2. `ADD x7, x5, x6` ne demektir?
3. Altı zarf biçiminin ortak yanı nedir?
4. Bir alan kullanılıyorsa yeri neden biçimden biçime değişmez?

## Yanıtlar

1. İşlem kodu.
2. x5 ile x6'yı topla, sonucu x7'ye yaz.
3. Hepsi 32 bit boyundadır.
4. İşlemci buyruğun türünü daha anlamadan onları okumaya başlayabilsin diye.



Bilge ve Yonga'nın Maceraları

# Buyrukların Dünyası



## Kitap 3.1a: Dediğimi Yap

Program nedir; kesin, sıralı adımlar ve makinenin tahmin etmemesi



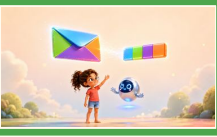
## Kitap 3.1b: İki Dünyanın Köprüsü

Buyruk kümesi mimarisi ve Getir-Çöz-Yürüt



## Kitap 3.2: Sıranın Üstündeki Kalemler

Yazmaçlar, bellek düzeni ve bayt sırası



## >> Kitap 3.3: Zarfın Üstündeki Bölmeler

Buyruk biçimleri ve adresleme kipleri



## Kitap 3.4: Parkta Kavşak

Dallanma, döngüler ve altıyordamlar



## Kitap 3.5: Mektup Fabrikası

Derleme zinciri: derleyici, çevirici, bağlayıcı



## Kitap 3.6: Toplama Makinesi

Aritmetik buyruklar: toplama, çıkarma, çarpma



## Kitap 3.7: Sihirli Maskeler

Mantık buyrukları: VE, VEYA, DEĞİL



## Kitap 3.8: Sıfırın Gücü

x0 yazmacı ve sıfırın gücü



## Kitap 3.9: Kulelerin Oyunu

Yığıt ve altıyordamlar



## Kitap 3.10a: Taşıyıcılar

Veri aktarma buyrukları (YÜKLE/SAKLA, İngilizcesi load/store): bellekten yazmaca, yazmaçtan belleğe



## Kitap 3.10b: Programın Bellekteki Mahallesi

Çalışan bir programın bellekteki yerleşimi: buyruk, veri, yığın ve yığıt



## Kitap 3.11: Dedektif Bilge ve Kayıp Sonuç

Hata ayıklama: ara nokta, adım adım yürütme

Bilge ve Yonga'nın yeni maceraları yolda!



# Bilge ve Yonga'nın Tüm Maceraları

Her kitap tek bir fikri, baştan sona bir hikâyeyle anlatır

## Kumdan Bilgisayara

1. Cilt · 15 kitap



## Hız ve Güç

2. Cilt · 10 kitap



## Buyrukların Dünyası

3. Cilt · 13 kitap



## İşlemcinin İçi

4. Cilt · 13 kitap



Bütün kitaplar ücretsiz, çevrimiçi:  
[bilgeveyonga.oguzergin.net](http://bilgeveyonga.oguzergin.net)



# KÜNYE

© 2026 Prof. Dr. Oğuz Ergin

**Zarfın Üstündeki Bölmeler**

Bilge ve Yonga: Bilgisayar Mimarisi Çocuk Kitapları Serisi

Ankara, 2026

Sürüm 1.1, 11 Ağustos 2026

Bu eser, Creative Commons Atıf-GayriTicari-Türetilemez 4.0 Uluslararası (CC BY-NC-ND 4.0) lisansı ile açık erişime sunulmuştur.

Lisans metni: <https://creativecommons.org/licenses/by-nc-nd/4.0/deed.tr>

## LİSANSIN KAPSAMADIĞI TÜM HAKLAR SAKLIDIR.

“Bilge ve Yonga” seri adı, “Bilge” ve “Yonga” karakter adları ile figürleri, seri amblemi, marka hakları, eser sahibinin adı ve unvanı ile manevi haklar bu lisansın kapsamı dışındadır ve ayrı yazılı izne bağlıdır.

Metin ve veri madenciliği ile yapay zekâ modeli eğitimi bakımından haklar açıkça saklı tutulmuştur.

Eserler olduğu gibi sunulmaktadır; belirli bir amaca uygunluk konusunda garanti verilmez.

Ticari kullanım izni ve sorularınız için: [bilgi@oguzergin.net](mailto:bilgi@oguzergin.net)

Telif ve lisans politikası: [bilgeveyonga.oguzergin.net/telif-ve-lisans.html](http://bilgeveyonga.oguzergin.net/telif-ve-lisans.html)

Atıf: Ergin, O. (2026). Bilge ve Yonga: Bilgisayar Mimarisi Çocuk Kitapları Serisi. <https://bilgeveyonga.oguzergin.net>

Bilge ve Yonga © 2026 Prof. Dr. Oğuz Ergin · CC BY-NC-ND 4.0



# Bilge ve Yonga'nın Maceraları

Buyrukların Dünyası

Prof. Dr. Oğuz Ergin

© 2026 · CC BY-NC-ND 4.0